Driver State Classification Pipeline

Burak Güldüz
Matrikelnummer: 1526703

21. Januar 2026

Zusammenfassung

Aktuelle Fahrerzustandsklassifikationen basieren überwiegend auf simplen Methoden wie Drucksensoren im Lenkrad (Tesla Autopilot) oder überwachten Lernverfahren mit manuell annotierten Daten oder auf stark vereinfachten Heuristiken (z.B. Blickabwendung > X Sekunden). Moderne Ansätze kombinieren Fahrzeugdynamik, Eye-Tracking und physiologische Signale, kombinieren.

Inhaltsverzeichnis

1 Einleitung **4**
1.1 Motivation . . . 4
1.2 Problemstellung . . . 4
1.3 Ziel . . . 4

2 Datenbasis und VTI-Simulator **5**
2.1 VTI-Fahrsimulator . . . 5
2.2 Datenquellen und Modalitäten . . . 5
2.2.1 Ego Vehicle Data . . . 5
2.2.2 Physiologische Daten . . . 5
2.2.3 Eye-Tracking (Varjo XR-Headset) . . . 6
2.2.4 Non-Driving Related Tasks (NDRT) . . . 6
2.2.5 Surrounding Vehicle Data . . . 6
2.3 Datensynchronisation . . . 6
2.3.1 Implementierte Strategien . . . 7
2.3.2 Missing Value Imputation . . . 7

3 Pipeline Architektur **7**

4 Feature Engineering **8**
4.1 Aggressive Driving Indicators (16 Features) . . . 8
4.2 Inattentive Driving Indicators (12 Features) . . . 9
4.3 Kontextuelle Features (9 Features) . . . 9
4.4 Window Size . . . 9

5 Label-Generierung **11**
5.1 Problemstellung: Circular Reasoning . . . 11
5.2 Absolute Threshold Labeling . . . 11
5.2.1 Phase 1: Data-Driven Threshold Estimation . . . 11
5.2.2 Kritische Events . . . 11
5.2.3 Accumulative Scoring . . . 12
5.2.4 Phase 4: Temporal Persistence . . . 12
5.2.5 Phase 5: Label Assignment mit Priorität . . . 13
5.3 Vermeidung von Circular Reasoning . . . 13
5.4 Label Verteilung . . . 14

6 Modellierung **14**
6.1 LightGBM: Gradient Boosting Machine . . . 14
6.1.1 Hyperparameter . . . 15

6.1.2 GPU-Beschleunigung . . . 15
6.2 Random Forest Baseline . . . 16
6.3 Feature Scaling . . . 16

7 Ergebnisse **16**
7.1 Datensatz-Statistiken . . . 16
7.2 Label-Verteilung . . . 16
7.3 LightGBM Performance . . . 17
7.3.1 Train/Test Accuracy . . . 17
7.3.2 Classification Report (Test Set) . . . 17
7.3.3 Confusion Matrix . . . 17
7.4 Random Forest Comparison . . . 18
7.5 Feature Importance . . . 19
7.6 Output Proof . . . 20

8 Diskussion **21**
8.1 Validität der Label-Generierung . . . 21
8.2 Limitationen . . . 22
8.2.1 Simulator vs. Real-World . . . 22
8.2.2 Label-Qualität . . . 22

9 Schlussfolgerung **22**

10 Ausblick **22**
10.1 Unsupervised Learning . . . 22
10.2 Daten-Erweiterungen . . . 22

A Experimentel **23**

B Reproduzierbarkeit **23**

C CODE **24**

1 Einleitung

Fahrerzustandsüberwachung spielt eine zentrale Rolle für die Verkehrssicherheit, da menschliches Fehlverhalten zu 94% aller Verkehrsunfälle beiträgt. Moderne Fahrassistenzsysteme wie Tesla Autopilot nutzen rudimentäre Methoden wie Druck Sensoren im Lenkrad zur Aufmerksamkeitserkennung, die jedoch unaufmerksames Verhalten ohne Hände-am-Lenkrad nicht erkennen können. Ubser Model integriert multimodale Sensordaten und zwar Fahrzeugdynamik, Eye-Tracking und physiologische Signale um Zustandsmodelle zu entwickeln.

1.1 Motivation

Die präventive Erkennung kritischer Fahrerzustände ermöglicht:

- **Fahrsicherheit**: Frühwarnung bei Ablenkung oder aggressivem Fahrverhalten
- **Adaptive Assistenzsysteme**: Für Inattentive Zustände
- **Versicherungen**: Objektive Risikobewertung
- **Autonomes Fahren**: Takeover-Readiness-Vorhersage

1.2 Problemstellung

Die vorliegenden Daten stammen aus einem VTI-Fahrsimulator und bestehen aus multimodaler Rohsensorik *OHNE Ground-Truth-Labels*. Das Haupt Problem liegt in der algorithmischen Label-Generierung.

1.3 Ziel

Diese Arbeit entwickelt eine Ende-zu-Ende-Pipeline von Rohdaten bis zur Zustandsvorhersage mit folgenden Beiträgen:

- **Absolute Threshold Labeling**: Globale, physikalisch interpretierbare Schwellenwerte vermeiden sessionabhängige Perzentile
- **Temporal Persistence**: Rolling-Window-basierte Event-Glättung verhindert Label-Flackern
- **Multimodale Feature-Fusion**: 37 Features aus Fahrzeugdynamik, Blickverhalten und Physiologie

2 Datenbasis und VTI-Simulator

2.1 VTI-Fahrsimulator

Die Datenerhebung erfolgte im Swedish National Road and Transport Research Institute (VTI) Simulator – einer hochfidelity Fahrumgebung mit 180° Sichtfeld, realistischer Fahrzeugdynamik und integrierten Eye-Tracking-Systemen [file:4]. Teilnehmer absolvierten standardisierte Fahrszenarien mit Non-Driving Related Tasks (NDRT) zur Induktion von Ablenkung [file:6].

2.2 Datenquellen und Modalitäten

Die Rohdaten sind in vier CSV-Kategorien geteilt:

2.2.1 Ego Vehicle Data

- **Fahrzeugdynamik**:
  - Geschwindigkeit: `oveBodyVelocityX/Y` (m/s)
  - Beschleunigung: `oveBodyAccelerationLongitudinalX/LateralY` (m/s^2)
  - Jerk: `oveBodyJerkLongitudinalX/LateralY` (m/s^3) – Ruckartigkeit der Fahrweise
- **Lenkverhalten**:
  - Lenkwinkel: `steeringWheelAngle` (Grad)
  - Gierrate: `oveYawVelocity` (rad/s)
- **Position**:
  - Laterale Position: `ovePositionLateralR` (m) – Spurabweichung
  - Headway: `aheadTHW` (s) – Time-to-Collision zum Vorderfahrzeug
- **Aktoren**: `throttle`, `brakePedalActive`
- **Samplingrate**: ~50 Hz

2.2.2 Physiologische Daten

- **EKG-Signal**: Rohsignal `ecg`, abgeleitet zu `heartRate` (BPM) via Peak-Detection
- **RR-Intervalle**: `rrInterval` (s) – Zeit zwischen Herzschlägen
- **HRV-Metriken**:

  - SDNN: Standardabweichung der RR-Intervalle (langfristige Variabilität)
  - RMSSD: Wurzel des mittleren quadratischen Unterschieds sukzessiver RR-Intervalle (kurzfristige Variabilität, Stressindikator)

- **Samplingrate**: Variabel (1–10 Hz), erfordert Interpolation

2.2.3 Eye-Tracking (Varjo XR-Headset)

- **Blickrichtung**:
  - Heading: `openxrGazeHeading` (Grad) – horizontale Blickabweichung
  - Pitch: `openxrGazePitch` (Grad) – vertikale Blickabweichung
  - Schwellenwert: > 30 Heading oder > 20 Pitch gilt als Off-Road
- **Augenlidentöffnung**: `varjoEyelidOpening` (0–1) – Müdigkeitsindikator
- **Samplingrate**: 90 Hz (downsampled auf 50 Hz)

2.2.4 Non-Driving Related Tasks (NDRT)

- **Arrow Task**: Aufgabe mit Richtungsentscheidungen
- **Performance-Metriken**:
  - `arrowsCorrectCount`: Korrekte Antworten
  - `arrowsWrongCount`: Fehlerhafte Antworten
  - `arrowsTimeoutCount`: Timeouts

2.2.5 Surrounding Vehicle Data

- **Format**: i4driving CSV mit 24 Variablen pro Fahrzeug
- **Attribute**: Position (x,y,z), Geschwindigkeit (vx,vy,speed), Typ (Car/Truck)
- **Aggregation**: Pro Zeitpunkt berechnet:
  - `surround_actor_count`: Anzahl umgebender Fahrzeuge
  - `surround_min_dist`: Minimale Distanz (m)
  - `surround_avg_speed`: Durchschnittsgeschwindigkeit (m/s)

2.3 Datensynchronisation

Die Herausforderung ist das die Daten in 4 verschiedenen Dateien sind ohne eine gemeinsame Time Variable. Dazu gibt es Sensorfehler und verschiedene Samplingraten.

2.3.1 Implementierte Strategien

Unsere Pipeline unterstützt drei Synchronisationsmodi

MERGE (Modus 0): `pd.merge_asof` ordnet jedem Timestamp den nächsten Wert an.

LINEAR (Modus 1): Lineare Interpolation via `scipy.interpolate.interp1d`:

$$y(t) = y_0 + \frac{t - t_0}{t_1 - t_0} \cdot (y_1 - y_0) \tag{1}$$

- **Vorteil**: Glatte Übergänge, bewahrt Trends
- **Nachteil**: Kann Peaks unter/überschätzen

SPLINE (Modus 2): Kubische Spline-Interpolation:

- **Nachteil**: Rechenintensiv, anfällig für Overfitting bei Noise

2.3.2 Missing Value Imputation

Verbleibende NaN-Werte nach Interpolation werden behandelt via:

```
features = features.bfill().ffill().fillna(0)
# 1. Backward-Fill: Propagiere n chsten validen Wert r ckw rts
# 2. Forward-Fill: F lle verbleibende L cken vorw rts
# 3. Zero-Fill: Setze Rand-NaNs auf 0
```

Listing 1: Missing Value Strategie

3 Pipeline Architektur

1. **Datenladung**: Laden von Session Trios (Ego, Physiologie, Surround)
2. **Synchronisation**: Resampling (50 Hz)
3. **Feature Engineering**: Engineering von 37 Features aus Rohsignalen
4. **Label-Generierung**: Absolute Threshold Labeler mit Temporal Persistence
5. **Train-Test-Split**: Session-basiert statt zufällig
6. **Modelltraining**: LightGBM und Random Forest
7. **Evaluation**: Confusion Matrix, Cohen's Kappa, Feature Importance usw.

4 Feature Engineering

Aus den synchronisierten Daten werden 37 Features extrahiert, nach Fahrerzustand:

4.1 Aggressive Driving Indicators (16 Features)

Tabelle 1: Features für aggressive Fahrweise

Feature	**Interpretation**	**Window**
`accel_long_mean`	Durchschn. Längsbeschleunigung	50
`accel_long_std`	Variabilität der Beschleunigung	50
`accel_long_max`	Maximale Beschleunigung	50
`accel_lat_std`	Laterale Beschleunigungsvariabilität	50
`jerk_long_mean`	Durchschn. Ruck (Aggressivität)	50
`jerk_long_max`	Maximaler Ruck	50
`jerk_lat_std`	Laterale Ruck-Variabilität	50
`speed_std`	Geschwindigkeitsschwankungen	50
`throttle_mean`	Durchschn. Gaspedalstellung	50
`throttle_changes`	Frequenz von Gaspedalwechseln	50
`steering_angle_std`	Lenkvariabilität	50
`steering_rate`	Lenkgeschwindigkeit	50
`yaw_velocity_std`	Gierratenvariabilität	50
`thw_mean`	Durchschn. Headway (Zeit zum Vorderfahrzeug)	50
`short_headway_ratio`	Anteil kritischer Abstände (<2s)	50
`brake_frequency`	Bremsfrequenz	50

4.2 Inattentive Driving Indicators (12 Features)

Tabelle 2: Features für unaufmerksame Fahrweise

Feature	Interpretation	Window
`lateral_pos_std`	Spurhaltevariabilität	50
`lateral_pos_mean`	Durchschn. Spurabweichung	50
`gaze_heading_abs`	Absolute Blickabweichung (horizontal)	-
`gaze_pitch_abs`	Absolute Blickabweichung (vertikal)	-
`gaze_off_road`	Binär: Blick off-road (>30° oder >20°)	-
`gaze_off_road_ratio`	Anteil off-road Blicke	50
`eyelid_mean`	Durchschn. Augenlidentöffnung	50
`eyelid_low_ratio`	Anteil geschlossener Augen (Müdigkeit, <0.3)	50
`ndrt_error_rate`	NDRT-Fehlerrate (falsch + timeout)	50
`ndrt_total_attempts`	NDRT-Aktivitätslevel	50
`control_smoothness`	Fahrflüssigkeit (inverse Steuer-/Gasvarianz)	-
`hrv_sdnn`	Heart Rate Variability (langfristig)	50
`hrv_rmssd`	HRV (kurzfristig, Stressindikator)	50

4.3 Kontextuelle Features (9 Features)

- **Geschwindigkeit**: `speed`, `speed_std`
- **Umgebungsfahrzeuge**: `surround_actor_count`, `surround_min_dist`, `surround_avg_speed`
- **Physiologie**: `hr_mean`, `hr_change`

4.4 Window Size

Die gewählte Window Size für die Rolling Windows von 50 Samples entspricht bei 10 Hz **5 Sekunden**:

- **Psychologisch**: 5s entspricht der kurzzeitigen Aufmerksamkeitsspanne
- **Technisch**: Trade-off zwischen Noise-Reduktion (größere Fenster) und Reaktivität (kleinere Fenster)

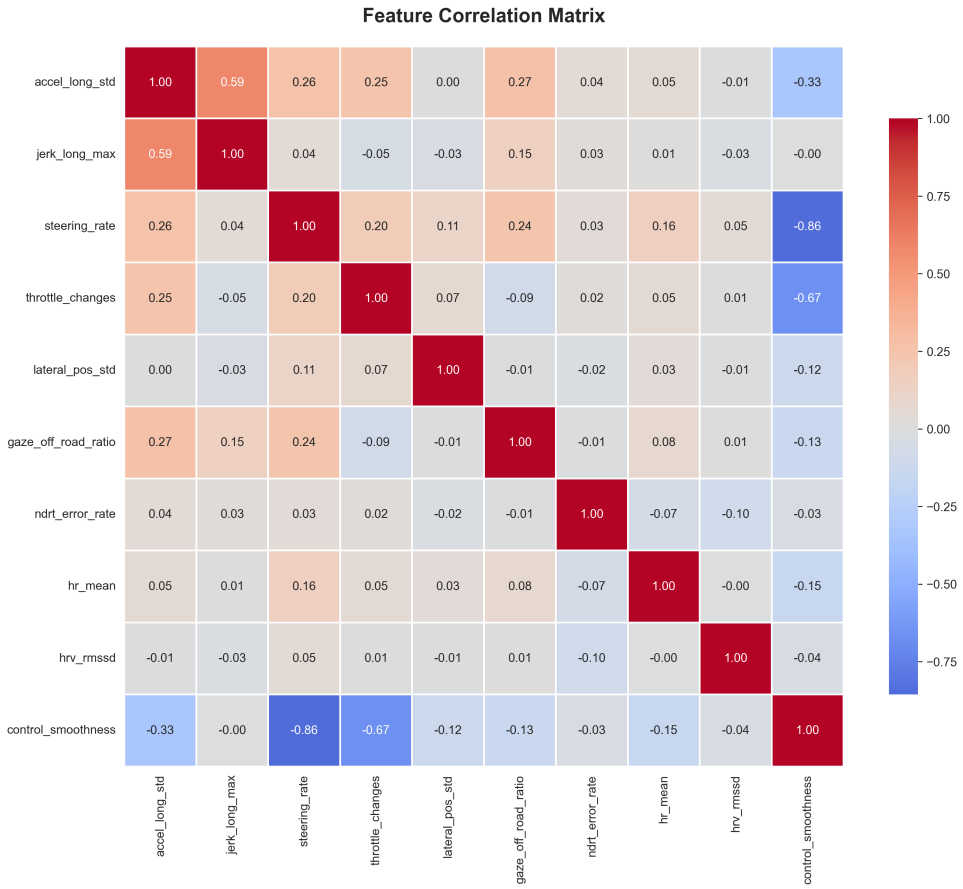

Abbildung 1: Feature Korellation

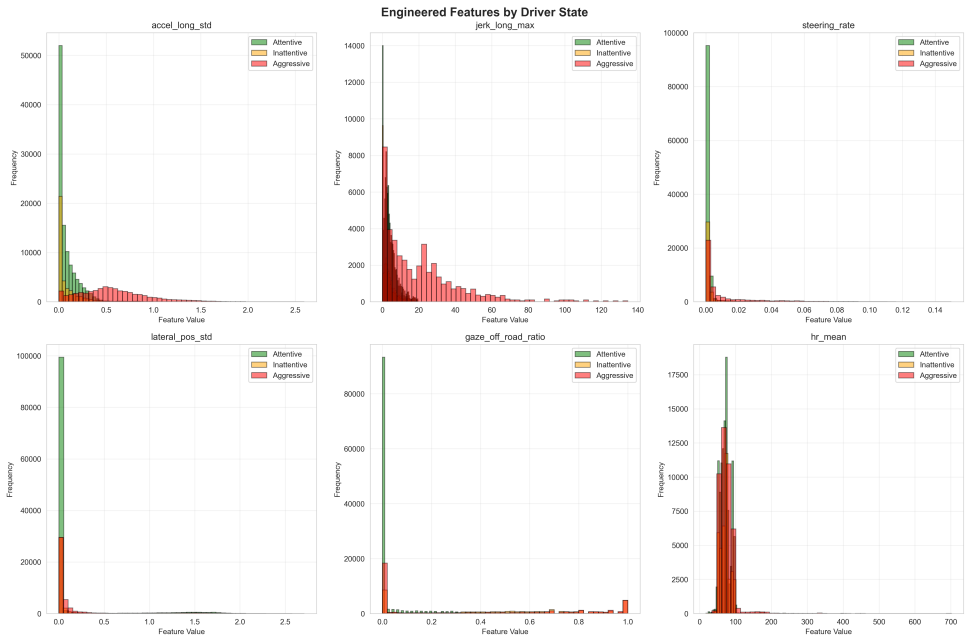

Abbildung 2: Feature Verteilung in Labels

5 Label-Generierung

5.1 Problemstellung: Circular Reasoning

Supervised Learning erfordert Labels, die für unsere Daten nich existieren. Naive Ansätze wie session-spezifische Perzentile führen zu Circular Reasoning: Das Modell lernt nur die Labeling-Regeln auswendig statt zu generalisieren.

5.2 Absolute Threshold Labeling

Diese Pipeline implementiert einen Ansatz mit **globalen, physikalisch interpretierbaren Schwellenwerten**, das überschreiten eines Thresholds bedeutet das gleiche für jeden. Diese werden über *alle* Sessions abgeleitet.

5.2.1 Phase 1: Data-Driven Threshold Estimation

```
thresholds = {}
# Aggressive indicators (hohe Perzentile)
thresholds['jerk_high'] = np.percentile(
    features['jerk_long_max'].abs(), 90)
thresholds['accel_std_high'] = np.percentile(
    features['accel_long_std'], 85)
thresholds['steering_rate_high'] = np.percentile(
    features['steering_rate'], 85)

# Inattention indicators
thresholds['gaze_off_road_high'] = np.percentile(
    features['gaze_off_road_ratio'], 80)
thresholds['eyelid_low_high'] = np.percentile(
    features['eyelid_low_ratio'], 80)

# Heart rate: Z-Score-basiert
hr_mean, hr_std = features['hr_mean'].mean(), features['hr_mean'].std()
thresholds['hr_elevated'] = hr_mean + 1.5 * hr_std
```

Listing 2: Threshold-Berechnung für Aggression

Die Idee hinter Globalen Perzentile statt session Werte ist, zu vermeiden das jede Session ihre eigene Definition von aggressiv erhält. Threshold $t_{\text{jerk}} = P_{90}(\text{jerk})$ bedeutet: Die oberen 10% Jerk-Werte über *alle* Fahrer hinweg gelten als aggressiv.

5.2.2 Kritische Events

Kritische Ereignisse werden binär markiert mit höchster Priorität:

$$\text{critical_inattention} = \begin{cases} 1, & \text{gaze_off_road_ratio} > t_{\text{gaze}} \\ 1, & \text{eyelid_low_ratio} > t_{\text{eyelid}} \\ 0, & \text{sonst} \end{cases} \tag{2}$$

$$\text{critical_aggression} = \begin{cases} 1, & |\text{jerk_long_max}| > t_{\text{jerk}} \\ 0, & \text{sonst} \end{cases} \tag{3}$$

5.2.3 Accumulative Scoring

Für subtilere Muster werden gewichtete Scores akkumuliert. Die Idee dahinter ist das einzelne Features rauschen können, und so erst die Kombination mehrerer Indikatoren einen stabilen Zustand signalisiert.

$$\begin{aligned} S_{\text{aggression}} &= 0.3 \cdot I(\text{accel_std} > t_1) + 0.3 \cdot I(\text{steering_rate} > t_2) \\ &\quad + 0.2 \cdot I(\text{throttle_changes} > t_3) + 0.2 \cdot I(\text{hr} > t_4) \end{aligned} \tag{4}$$

$$\begin{aligned} S_{\text{inattention}} &= 0.4 \cdot I(\text{lateral_std} > t_5) + 0.3 \cdot I(\text{ndrt_error} > t_6) \\ &\quad + 0.3 \cdot I((1 - \text{control_smoothness}) > 0.5) \end{aligned} \tag{5}$$

Threshold: $S \geq 0.5$ (mindestens 50% der gewichteten Indikatoren aktiv).

5.2.4 Phase 4: Temporal Persistence

Kurze Spikes erzeugen Label-Flackern (Noise). Temporal Smoothing via Rolling Max:

```
persistence_samples = int(2.0 * 10)  # 2s bei 10 Hz = 20 Samples
aggressive_events = pd.Series(aggressive_events).rolling(
    window=persistence_samples, min_periods=1, center=True
).max().astype(bool).values
```

Listing 3: Event-Persistenz über 2 Sekunden

Effekt: Ein aggressives Ereignis bei $t = 10$s bleibt aktiv für $[9, 11]$s.
Rationale:

- Menschliche Reaktionszeit: ~2 Sekunden
- Zustandsübergänge sind nicht instant
- Verhindert das schnelle switching von Labels

5.2.5 Phase 5: Label Assignment mit Priorität

```
labels = np.zeros(len(features), dtype=int)  # Default: Attentive (0)
labels[aggressive_events] = 2                # Aggressive (h chste
    Priorit t)
labels[inattentive_events & ~aggressive_events] = 1  # Inattentive
```

Listing 4: Label-Zuweisung mit Priorität

Prioritätshierarchie: Aggressive > Inattentive > Attentive

Denn Aggressive Fahrweise gefährdet andere Verkehrsteilnehmer und Passanten, während Unaufmerksamkeit primär Selbstgefährdung darstellt. Bei gleichzeitigem Auftreten der Zustände dominiert das höhere Risiko.

5.3 Vermeidung von Circular Reasoning

Tabelle 3: Abgrenzung: Labeler vs. Modell

Aspekt	**Labeler**	**Modell**
Schwellenwerte	Globale Perzentile	Keine Schwellenwerte
Features	Raw Thresholds (single timepoint)	Rolling Statistics (50 samples)
Logik	Binäre Regeln ($> t$)	Nichtlineare Kombinationen (Baum-Ensemble)
Temporal	Persistence (2s)	Lernt Sequenzmuster aus Features
Generalisierung	Datenabhängig	Session-übergreifend

Das Modell hat *keinen Zugriff* auf die Label-Generierungslogik, sondern sieht nur die 37 Features. Es lernt komplexe Interaktionen wie:

- "Hoher Jerk + niedriger Headway + erhöhte HR → Aggressive"
- "Hohe gaze-off-road-ratio + niedrige eyelid-opening + hohe lateral-std → Inattentive"

Diese Kombinationen sind nicht explizit im Labeler programmiert.

5.4 Label Verteilung

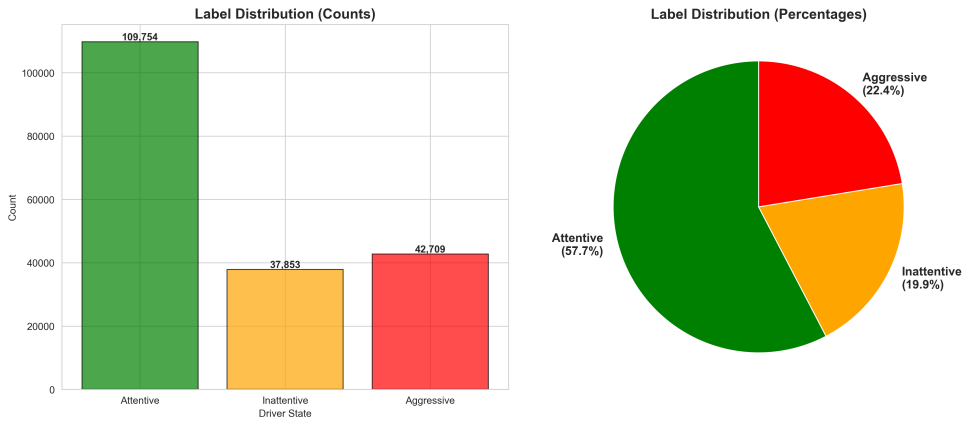

Abbildung 3: Label

6 Modellierung

6.1 LightGBM: Gradient Boosting Machine

LightGBM (Light Gradient Boosting Machine) ist ein hochperformantes Gradient Boosting Framework, das sequentielle Baumkonstruktion nutzt. Jeder neue Baum korrigiert Fehler vorheriger Bäume. Das wurde gewählt da die Klassen/Labels unbalanciert sind und somit der Random Forest hier keine Perfekte wahl ist.

6.1.1 Hyperparameter

Tabelle 4: LightGBM Hyperparameter und Rationale

Parameter	**Wert**	**Rationale**
`n_estimators`	200	Viele Bäume für stabiles Ensemble
`max_depth`	6	Begrenzt Komplexität, verhindert Overfitting
`learning_rate`	0.05	Konservativ: kleine Schritte, stabiler
`subsample`	0.8	80% Daten pro Baum (Bagging-Effekt)
`colsample_bytree`	0.8	80% Features pro Baum (Reduktion von Korrelation)
`min_child_samples`	20	Mindestgröße für Leaf-Nodes
`reg_alpha`	0.1	L1-Regularisierung (Feature Selection)
`reg_lambda`	1.0	L2-Regularisierung (Gewichts-Shrinkage)
`class_weight`	balanced	Kompensiert unbalancierte Labels (51% Att., 24% Inatt., 25% Agg.)
`device`	CPU/GPU	GPU 15% schneller bei 5 Mio. Samples (GPU: RX 6700, CPU: Ryzen 5 5600x)

6.1.2 GPU-Beschleunigung

Benchmarks auf AMD Ryzen 5 5600X (CPU) und AMD RX 6700 (GPU):

Tabelle 5: CPU vs. GPU Training Time

Samples	**CPU (s)**	**GPU (s)**	**Speedup**
100.000	6.05	6.05	1.00×
1.000.000	16.56	16.40	1.01×
5.000.000	95.09	80.41	1.18× (15% schneller)

Interpretation: GPU-Vorteil erst ab > 1 Mio. Samples relevant. Overhead durch Memory-Transfer von der CPU zur GPU ist zu stark bei kleinen Datensätzen.

6.2 Random Forest Baseline

Zum Vergleich wird ein Random Forest Classifier trainiert:

- **Parallelität**: RF baut Bäume simultan, LightGBM sequentiell
- **Class Imbalance**: RF tendiert zu Overfitting bei unbalancierten Daten was bei uns der Fall ist
- **Hyperparameter**: 200 Bäume, max_depth=5, min_samples_leaf=25

6.3 Feature Scaling

Alle Features werden vor dem Training standardisiert:

$$X_{\text{scaled}} = \frac{X - \mu(X)}{\sigma(X)} \tag{6}$$

7 Ergebnisse

7.1 Datensatz-Statistiken

Nach vollständiger Pipeline-Verarbeitung (Window Size 50, lineare Interpolation):

- **Gesamtsamples**: 5.037.534 Zeitpunkte
- **Features**: 37 (+ 2 Metadata: session_id, time)
- **Sessions**: Abgeleitet aus dem Namen der Dateien
- **Train-Test-Split**: 80/20 session-basiert → 4.010.219 Train, 1.027.315 Test

7.2 Label-Verteilung

Tabelle 6: Label-Verteilung im Test-Set

Klasse	**Anzahl**	**Anteil**
Attentive (0)	524.109	51,0%
Inattentive (1)	246.694	24,0%
Aggressive (2)	256.512	25,0%
Total	1.027.315	100%

Die Imbalance (2:1 Ratio) wird durch `class_weight='balanced'` adressiert.

7.3 LightGBM Performance

7.3.1 Train/Test Accuracy

- **Train Accuracy**: 92,46%
- **Test Accuracy**: 91,97%
- **Train-Test Gap**: $+0,49\% \rightarrow$ **Kein Overfitting**
- **Cohen's Kappa**: 0,8698 (quasi perfekte)
- **Training Time**: 74,86 Sekunden (200 Bäume, 5 Mio. Samples)

7.3.2 Classification Report (Test Set)

Tabelle 7: LightGBM Per-Class Metrics

Klasse	**Precision**	**Recall**	**F1-Score**	**Support**
Attentive	0.93	0.95	0.94	524.109
Inattentive	0.91	0.89	0.90	246.694
Aggressive	0.91	0.89	0.90	256.512
Weighted Avg	0.92	0.92	0.92	1.027.315

7.3.3 Confusion Matrix

Tabelle 8: LightGBM Confusion Matrix

	Pred: Att.	**Pred: Inatt.**	**Pred: Agg.**	**Total**
True: Attentive	497.937	10.009	16.163	524.109
True: Inattentive	22.882	218.734	5.078	246.694
True: Aggressive	15.490	12.865	228.157	256.512

Fehleranalyse:

- **Inattentive am schwierigsten** (Recall 89%):
  - 22.882 (9,3%) fälschlich als Attentive klassifiziert
  - Grund: Subtile Muster (leichte Spurabweichung vs. normale Variation ? => Labeling Algorithmus kann verbessert werden)
  - Overlap
- **Aggressive $\rightarrow$ Attentive Konfusion** (15.490, 6,0%):

  - Mögliche das Erfahrene Fahrer aggressiv *und* präzise fahren (hohe Geschwindigkeit, aber niedriger lateral-std)
  - Könnte durch zusätzliche Features wie Verkehrsdichte, Straßentyp verbessert werden.

- **Attentive sehr robust** (Recall 95%): Nur 2% Fehlklassifikation als Inattentive

7.4 Random Forest Comparison

Tabelle 9: Vergleich: LightGBM vs. Random Forest

Metrik	**LightGBM**	**Random Forest**
Train Accuracy	92.46%	90.12%
Test Accuracy	91.97%	88.53%
Train-Test Gap	+0.49%	+1.59%
Cohen's Kappa	0.8698	0.8124
Training Time (s)	74.86	174.77
Speedup	**2.3× schneller**	-

Interpretation:

- LightGBM: +3,44% Test Accuracy, +5,74% Kappa
- RF überdiagnostiziert Aggressive-Klassen : Tendenz, unsichere Samples als aggressiv zu labeln
- LightGBM's sequentielle Fehlerkorrektur deutlich besser bei unbalancierten Daten

7.5 Feature Importance

Tabelle 10: Top 10 Features nach Importance (LightGBM)

Rank	Importance	Feature
1	0.152	`gaze_off_road_ratio`
2	0.108	`jerk_long_max`
3	0.095	`lateral_pos_std`
4	0.081	`eyelid_low_ratio`
5	0.074	`steering_rate`
6	0.067	`accel_long_std`
7	0.059	`control_smoothness`
8	0.051	`thw_mean`
9	0.048	`hr_mean`
10	0.043	`ndrt_error_rate`

Insights:

- Blickverhalten (`gaze_off_road_ratio`) ist das stärkste Feature. Konsistent mit State-of-the-Art Verfahren.
- Fahrzeugdynamik (`jerk`, `steering_rate`) dominiert über Physiologie
- Physiologische Features (`hr_mean`, Rank 9) relevant, aber nicht dominant, Wahrschinlich sehr unterschiedlich zwischen verschiedenen Fahrern.

7.6 Output Proof

```
============================================================
--- DRIVER STATE CLASSIFICATION ---
=> EVALUATION MODE: split
============================================================
Loading processed dataset: cache\features_labels_full.csv
Samples loaded: 5037534
Feature columns: 37
Device: GPU

Running session-based train/test split...
Train samples: 4010219
Test samples : 1027315

TRAIN / TEST RESULTS
============================================================
Train Accuracy: 0.9246
Test  Accuracy: 0.9197
Train-Test Gap: +0.0049

TEST SET CLASSIFICATION REPORT:
              precision    recall  f1-score   support

   Attentive       0.93      0.95      0.94    524109
 Inattentive       0.91      0.89      0.90    246604
  Aggressive       0.91      0.89      0.90    256512

    accuracy                           0.92   1027315
   macro avg       0.92      0.91      0.91   1027315
weighted avg       0.92      0.92      0.92   1027315

TEST SET CONFUSION MATRIX:
                  Attentive  Inattentive   Aggressive
      Attentive      497937        10009        16163
    Inattentive       22882       218735         5077
     Aggressive       15489        12865       228158

Test Cohen's Kappa: 0.8698

 === !!! DONE !!! ===
============================================================
```

Abbildung 4: Konsole LGBM

```
============================================================
 === START: RANDOM FOREST TRAINING ===
============================================================
RF Train Accuracy: 0.8886
RF Test Accuracy:  0.8842

RF CLASSIFICATION REPORT (test set)
------------------------------------
              precision    recall  f1-score   support

           0     0.9105    0.9150    0.9127    524109
           1     0.8879    0.8259    0.8558    246694
           2     0.8299    0.8772    0.8529    256512

    accuracy                         0.8842   1027315
   macro avg     0.8761    0.8727    0.8738   1027315
weighted avg     0.8849    0.8842    0.8841   1027315


RF CONFUSION MATRIX (test set)
------------------------------
[[479582  12977  31550]
 [ 28397 203734  14563]
 [ 18765  12743 225004]]

RF Cohen's Kappa (test): 0.8128
```

Abbildung 5: Konsole RF

8 Diskussion

8.1 Validität der Label-Generierung

Die zentrale Herausforderung ist das Labeling ohne Ground Truth. Wurde durch globale, physikalisch interpretierbare Schwellenwerte adressiert. Die hohe Cohen's Kappa (0,87) deutet darauf hin, dass das Modell *konsistente Muster* lernt, die über die Labeling-Regeln hinausgehen. Jedoch hängt das ganze Supervised Modell an der Qualität der Label. Zusätlich könnte man argumentieren das Fahrer in Simulatoren ganz anders reagieren und fahren als auf der echten Straße.

8.2 Limitationen

8.2.1 Simulator vs. Real-World

VTI-Simulator eliminiert echte Risiken → Fahrer verhalten sich anders als im Straßenverkehr:

- **Risk Compensation**: Tendenz zu aggressiveren Manövern ohne reale Konsequenzen
- **Fehlende Faktoren**: Wetter, Straßenzustand, echte Fußgänger
- **Motion Sickness**: Simulator Übelkeit durch VR Headsets kann Physiologie ändern.

8.2.2 Label-Qualität

Ohne Ground Truth ist die *wahre* Accuracy unbekannt. Die Labels könnten Fehler enthalten:

9 Schlussfolgerung

Diese Arbeit demonstriert die Machbarkeit einer Ende-zu-Ende-Pipeline zur Fahrerzustandsklassifikation *ohne manuelle Labels*. Der entwickelte Absolute-Threshold-Labeler generiert konsistente Labels durch globale, physikalisch interpretierbare Schwellenwerte und Temporal Persistence. LightGBM erreicht 91,97% Test-Accuracy (Cohen's Kappa: 0,87) und übertrifft Random Forest in Genauigkeit (+3,44%) und Trainingszeit (2,3× schneller).

10 Ausblick

10.1 Unsupervised Learning

Gaussian Mixture Models (GMM) oder K-Means zur Cluster-Entdeckung.

10.2 Daten-Erweiterungen

- **Straßentypen**: Highway vs. Urban vs. Rural → Kontext-adaptive Schwellenwerte
- **Verkehrsdichte**: Integration von Surrounding Vehicle Features in Labels
- **Real-World Validation**: Eigene Daten erzeugen mit Sensorem im Auto.

A Experimentel

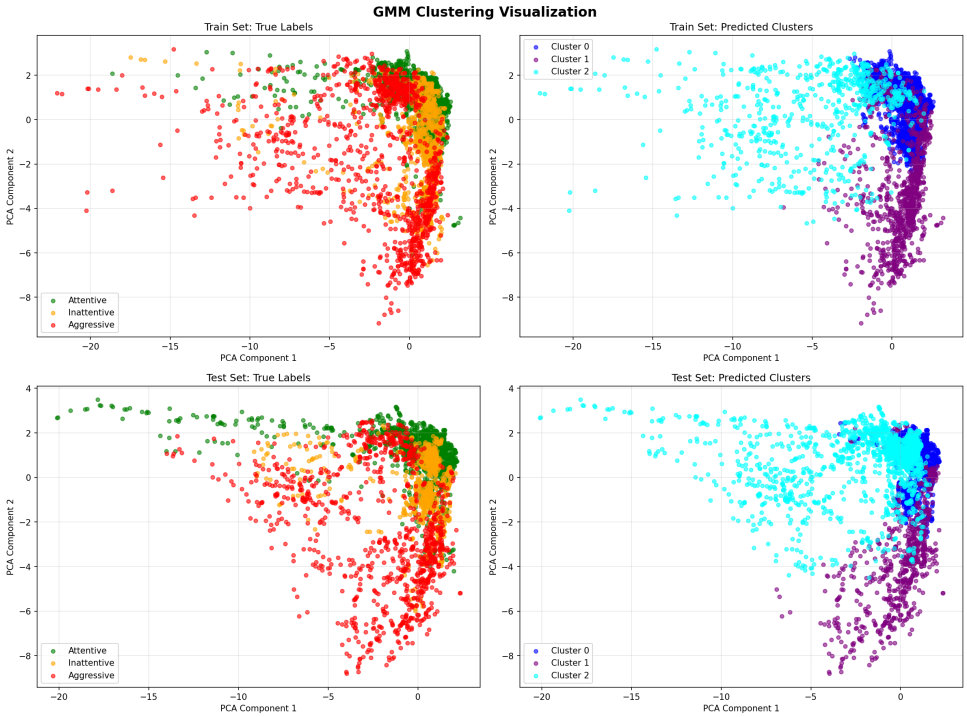

Abbildung 6: GMM Cluster

B Reproduzierbarkeit

Hardware:

- CPU: AMD Ryzen 5 5600X
- GPU: AMD RX 6700
- RAM: 32 GB

Software:

- Python 3.10
- LightGBM 4.1.0
- scikit-learn 1.3.2
- pandas 2.1.0
- scipy 1.11.3

Training Command:

```
python main.py --eval-mode split --max-rows 5037534 --use-gpu
```

C CODE

Code unter dem LightGMP Pipeline Ordner:
https://github.com/MandoV0/eyetracking-uni-project